مكتب

Lab Activity

Week 4: String Basics - Indexing, Slicing, and Methods

Duration: 55 minutes — Work in pairs!

Name: _____ **Partner:** _____

Lab Goals

By the end of this lab, you will be able to:

- Access individual characters in strings using indexing
- Extract portions of strings using slicing
- Apply string methods to transform and analyze text
- Debug common string operation errors
- Build simple text manipulation programs

Getting Started

1. Extract the given week4_lab.zip file to your desktop. This folder contains all the files you need for this lab.
2. Remember: Strings are like boxes of characters, and we count starting from 0!

1 Exercise 1: String Indexing Practice (8 minutes)

Let's explore how to access individual characters in strings!

Exercise 1: Understanding string positions

Open week4_ex1.py and complete the code:

```
# String indexing exploration
word = "PYTHON"

# Positive indexing - fill in the indices
print("Positive indexing:")
print(f"First character: {word[___]}")  # Should print 'P'
print(f"Third character: {word[___]}")  # Should print 'T'
print(f"Last character: {word[___]}")   # Should print 'N'

# Negative indexing - fill in the indices
print("\nNegative indexing:")
print(f"Last character: {word[___]}")     # Should print 'N'
print(f"Second to last: {word[___]}")     # Should print 'O'
print(f"First character: {word[___]}")    # Should print 'P'
```

Now complete these tasks:

- For the string "PYTHON", what is at index 4? _____
- What is at index -4? _____
- What happens if you try to access `word[10]`? Try it!
  Error type: _____

Challenge: Complete the code to check if a word starts and ends with the same letter:

```
name = input("Enter a word: ")
# Fill in the condition
if name[___] == name[___]:
    print("Starts and ends with same letter!")
else:
    print("Different start and end letters")
```

Tip: Remember: Index -1 always gives you the last character!

Checkpoint: Show your teacher your working code before moving on!

2 Exercise 2: String Slicing Mastery (10 minutes)

Time to extract portions of text using slicing!

Open week4_ex2.py and complete the slicing operations:

```
message = "HELLO WORLD"
print(f"Original: {message}")
print("-" * 30)

# Basic slicing - complete the indices
print(f"First 5 chars: {message[___:___]}")     # Get "HELLO"
print(f"Get WORLD: {message[___:___]}")         # Get "WORLD"

# Omitting start or end - fill these in
print(f"First 4 chars: {message[:___]}")        # Get "HELL"
print(f"From index 6: {message[___:]}")         # Get "WORLD"

# Using step - complete these
print(f"Every 2nd char: {message[::___]}")      # Get "HLOWRD"
print(f"Reversed: {message[___]}")              # Get "DLROW OLLEH"
```

Predict first, then test:

Before running, write what these will output:

- `message[2:7]` will give: _____
- `message[-5:-1]` will give: _____
- `message[1::3]` will give: _____

Slicing Practice - Fill in the correct slice notation:

```python
text = "PROGRAMMING"
# Get "GRAM" (indices 3-7)
part1 = text[___:___]

# Get last 4 characters
part2 = text[___:]

# Get every other letter starting from index 1
part3 = text[___::___]

print(part1, part2, part3)
```

Pair Programming: Switch who types every 5 minutes!

3 Exercise 3: String Methods Explorer (10 minutes)

Let's discover the power of built-in string methods!

Open week4_ex3.py and complete:

```python
# String methods testing
text = "  Python Programming  "

print("Original:", repr(text))  # repr shows spaces
print("-" * 40)

# Case methods - fill in the method names
upper_text = text._____()       # Make uppercase
lower_text = text._____()       # Make lowercase
title_text = text._____()       # Title case

print(f"Upper: {upper_text}")
print(f"Lower: {lower_text}")
print(f"Title: {title_text}")

# Cleaning and searching - complete these
clean = text._____()            # Remove spaces from both ends
position = text._____("Pro")    # Find position of "Pro"
count_n = text._____("n")       # Count how many 'n's
new_text = text._____("Python", "Java")  # Replace word

print(f"Cleaned: {clean}")
print(f"Position of 'Pro': {position}")
print(f"Count of 'n': {count_n}")
print(f"Replaced: {new_text}")
```

Build a Username Validator:

```python
# Username rules: 3-10 chars, no spaces, all lowercase
username = input("Create username: ")

# Complete the conditions
if len(username) < ___ or len(username) > ___:
    print("Must be 3-10 characters!")
elif " " ___ username:  # Check if space exists
    print("No spaces allowed!")
elif username != username._____():  # Check if lowercase
```

```
    print("Must be lowercase!")
else:
    print(f"Username '{username}' accepted!")
```

Checkpoint: Username validator working? Excellent!

4 Exercise 4: Word Games with Strings (12 minutes)

Let's build some fun text manipulation programs!

Program 1: Initials Extractor

Open week4_ex4a.py and complete:

```
# Extract initials from a name
print("=== Initials Extractor ===")
full_name = input("Enter your full name: ")

# Split into words
words = full_name._____()  # Which method splits by spaces?

# Build initials
initials = ""
for word in words:
    initials += word[___]  # Add first letter of each word

print(f"Your initials: {initials._____()}")  # Make uppercase
```

Program 2: Palindrome Checker

Open week4_ex4b.py and complete:

```
# Check if word reads same forwards and backwards
print("=== Palindrome Checker ===")

word = input("Enter a word: ")._____()  # Convert to lowercase
# Remove any spaces
clean_word = word._____(" ", "")  # Remove spaces

# Check if palindrome
if clean_word == clean_word[___]:  # How to reverse?
    print(f"'{word}' IS a palindrome!")
else:
    print(f"'{word}' is NOT a palindrome")
```

Test with: "racecar", "hello", "noon"

Program 3: Password Strength Checker

```
# Check password strength
password = input("Enter password: ")

# Complete the checks
has_upper = False
has_lower = False
has_digit = False
```

```
8
9  for char in password:
10     if char._____():  # Check if uppercase
11         has_upper = True
12     elif char._____():  # Check if lowercase
13         has_lower = True
14     elif char._____():  # Check if digit
15         has_digit = True
16
17 # Check length and variety
18 if len(password) < ___:  # Minimum 8 characters
19     print("Too short! Need 8+ characters")
20 elif not (has_upper and has_lower and has_digit):
21     print("Needs uppercase, lowercase, and number!")
22 else:
23     print("Strong password!")
```

5 Exercise 5: String Detective - Debug Challenge (10 minutes)

These programs have bugs. Find and fix them!

Bug 1: Index Confusion

```
1 # Should print first and last character of any input
2 text = input("Enter text: ")
3 print(f"First: {text[1]}")    # Bug here!
4 print(f"Last: {text[-0]}")    # Bug here too!
```

Your fixes:

- Line 3: Change `[1]` to `[___]`
- Line 4: Change `[-0]` to `[___]`

Bug 2: Slicing Error

```
1 # Should extract "LOVE" from "I LOVE CODING"
2 sentence = "I LOVE CODING"
3 word = sentence[1:4]  # This gives " LO" not "LOVE"
```

Correct slice: `sentence[___:___]`

Bug 3: Method Misunderstanding

```
1 # Should make text uppercase and store it
2 message = "hello world"
3 message.upper()  # Bug: doesn't save result!
4 print(message)   # Still prints "hello world"
```

Fix (complete the line): `message = _____`

Bug 4: Find Method Logic

```
1 # Should check if @ exists in email
2 email = input("Enter email: ")
3 if email.find("@") == True:  # Bug: find returns index!
4     print("Has @ symbol")
```

What does `find()` return when not found? _____

Correct condition: `if email.find("@") ___ ___:`

Checkpoint: All bugs fixed? Great debugging!

6 Exercise 6: String Transformations (10 minutes)

Practice combining multiple string operations!

Open week4_ex6.py and complete:

```
# Text transformer program
print("=== Text Transformer ===")
text = input("Enter text to transform: ")

# Transform 1: Alternating case
alt_case = ""
for i in range(len(text)):
    if i % 2 == 0:
        alt_case += text[i]._____()  # Make even indices upper
    else:
        alt_case += text[i]._____()  # Make odd indices lower

print(f"Alternating: {alt_case}")

# Transform 2: First and last of each word
words = text._____()
first_last = ""
for word in words:
    if len(word) >= 2:
        first_last += word[___] + word[___] + " "
    else:
        first_last += word + " "

print(f"First+Last: {first_last._____()}")  # Remove trailing space

# Transform 3: Vowel counter
vowels = "aeiouAEIOU"
vowel_count = 0
for char in text:
    if char ___ vowels:  # Check if char is a vowel
        vowel_count += 1

print(f"Vowel count: {vowel_count}")
```

Checkpoint: All transformations working? Excellent!

7 Lab Summary

Checkpoint: Final checkpoint - make sure you have:

- ☐ Practiced string indexing with positive and negative indices
- ☐ Mastered string slicing with different patterns
- ☐ Used string methods for case conversion and searching
- ☐ Built word games using string operations
- ☐ Fixed string-related bugs
- ☐ Completed string transformation exercises
- ☐ Worked well with your partner
- ☐ Shown all checkpoints to your teacher

Reflection (2 minutes)

Rate your understanding of today's concepts:

Concept	Need Help	Getting It	Got It!
String indexing [0] [-1]	○	○	○
String slicing [start:end]	○	○	○
String methods (.upper(), .find())	○	○	○
Debugging string errors	○	○	○

What's the most useful string operation you learned today?

_____

Which was trickier: indexing, slicing, or methods? Why?

_____

Great job manipulating text with strings!